

CONVERA Project Report

Purpose

CONVERA is a local LLM inference runtime focused on reducing repeated work across runs. The public project is intentionally a lite open-core runtime. It provides a real execution loop, local model loading, persistent reusable state, benchmark tools, a FastAPI dashboard, and a privacy-first telemetry contract.

The public repository does not include private acceleration systems. It exposes a narrow public API boundary and keeps internal research and non-public optimization work out of the public tree.

What The Project Does

CONVERA runs Hugging Face causal language models from a local model directory. During inference, it records high-level runtime state and stores reusable artifacts in a local content-addressed tensor store. On repeated prompts, the runtime can reuse persisted KV state and report whether a request was served with a cache hit.

The project also includes:

- A command line interface.
- A local web dashboard.
- A benchmark harness.
- A metrics-only telemetry payload builder.
- A public repeated-prompt benchmark.
- A server telemetry handoff document.
- A public/private boundary document.
- A public execution inspector.
- Optional audit trace export.

Current Public Systems

Model Loading

Model loading lives in `core/model_loader.py`. It loads a tokenizer and `AutoModelForCausalLM` from the configured local model path. The default model path is:

```
models/llama3
```

The loader uses Hugging Face Transformers and can use automatic device mapping when supported by the local environment.

Inference Engine

The inference engine lives in `inference/engine.py`.

Main responsibilities:

- Tokenize the prompt.
- Move inputs to the selected backend.
- Check persistent KV state.
- Run generation.
- Store prompt KV state after execution.
- Return decoded text and runtime stats.

The returned stats include:

- Latency.
- Tokens generated.
- Tokens per second.
- KV cache hit status.
- Backend name.
- Memory use.
- Requested and actual precision.
- Tokens reused.
- Tokens computed.
- Compute avoided percentage.
- Execution route mode.
- Validation status and validation latency.
- Fallback reason, when applicable.

KV Manager

The KV manager lives in `inference/kv_manager.py`.

Main responsibilities:

- Hash prompt token IDs.
- Look up previously stored KV state.
- Persist KV tensors through the CONVERA tensor store.
- Track hit and miss counts.
- Maintain a small in-process cache for recently used state.

CONVERA Store Lite

The lite store lives in `convera_store_lite/`.

Main responsibilities:

- Split tensor bytes into fixed-size chunks.
- Hash chunks.
- Store compressed chunk payloads on disk.
- Reuse chunks when the same payload appears again.
- Reconstruct tensors exactly from stored manifests.

Default storage paths:

`data/chunks`

data/index

CONVERA Tensor Store

The tensor store facade lives in `convera_store/`.

It provides a higher-level interface for storing and loading tensors. It uses `convera_store_lite` under the public runtime.

Public API Boundary

The public API boundary lives in `convera_core_api/interface.py`.

Exposed functions:

```
store_tensor(tensor)
load_tensor(refs)
optimize_kv(kv_tensor)
merge_states(state_a, state_b)
```

The public implementation returns tensors, references, and simple metadata only.

Backend Selection

Backend support lives in `core/`.

The project includes backend modules for:

- CPU.
- CUDA.
- MPS.
- ROCm-compatible PyTorch.

The selected backend is used by the inference engine for moving inputs and reporting runtime memory.

Runtime Precision

CONVERA includes a public multi-precision loading path. Supported requested modes are:

- fp16
- int8
- int4
- auto

Quantized modes use Hugging Face quantization loading paths when supported by the local hardware and installed dependencies. Unsupported modes fall back safely instead of crashing the runtime.

Execution Router

The public execution router lives in `inference/execution_router.py`.

It produces deterministic public route modes:

```
cache  
kv  
full
```

The router is intentionally simple in the public runtime. It reports what path was selected without exposing private decision logic, scoring formulas, or intermediate states.

Cache Validation Records

The public validation layer lives in `inference/cache_validator.py`.

It creates lightweight validation records for cached responses and validation-mode runs. Records include prompt and response hashes, timestamp, and method name. They do not include prompts, outputs, token traces, cache contents, or private runtime details.

Validation records are stored locally at:

```
data/index/validation_records.json
```

Validation mode can be enabled with:

```
CONVERA_VERIFICATION_MODE=1 uvicorn ui.app:app --reload
```

Adaptive Routing Model

The public adaptive routing model lives in `inference/routing_model.py`.

It learns from public-safe runtime outcomes:

- route mode
- cache hit status
- latency
- tokens reused
- tokens computed
- compute avoided percentage

The model is conservative. It only proposes public route modes after enough local history exists and otherwise falls back to deterministic routing. It does not expose private scoring formulas, private decision logic, prompts, outputs, or token traces.

Runtime outcome history is stored at:

```
data/learning_history.json  
data/routing_model.json
```

Execution Inspector

The public execution inspector lives in `inference/execution_inspector.py`.

It records the high-level request path:

- routing
- precision
- execution
- validation

Records are stored at:

`data/execution_records.json`

The UI can retrieve the latest record or a record by request ID.

Audit Trace Export

The audit system lives in:

`audit/audit_logger.py`
`audit/exporter.py`

Audit mode records redacted execution traces. Trace fields include request ID, prompt hash, route mode, precision, latency, token counts, compute avoided percentage, and validation status.

Audit traces do not include prompts, generated outputs, raw tokens, local paths, private formulas, or private runtime internals.

Enable audit mode for every request:

```
CONVERA_AUDIT_MODE=1 uvicorn ui.app:app --reload
```

The UI also supports per-request audit traces through the Audit trace checkbox.

Benchmarks

Benchmark tools live in:

`benchmarks/`
`metrics/benchmark_runner.py`

The public benchmark runner demonstrates repeated-prompt behavior using only public-safe measurements:

- Cache hit status.
- Latency in milliseconds.
- Response length.
- Benchmark JSON output.
- Benchmark graph output.

It does not expose token traces, cache contents, KV internals, or private mechanisms.

Generated benchmark artifacts are written to:

```
metrics/output/benchmark.json
metrics/output/benchmark.png
metrics/output/comparison.png
```

Telemetry

Telemetry lives in:

```
telemetry/convera_payload.py
telemetry/convera_client.py
```

Telemetry is opt-in. The client sends metrics only when these variables are configured:

```
CONVERA_METRICS_API_URL
CONVERA_METRICS_API_KEY
```

The payload type is:

```
convera-metrics-only
```

Telemetry does not send prompts, outputs, file names, local paths, model weights, token traces, or cache contents.

Local UI

The UI is served through FastAPI in `ui/app.py` and `ui/static/index.html`.

The UI provides:

- Prompt input.
- Run button.
- Output display.
- Metrics cards.
- Recent run history.
- Refresh button.
- Benchmark graph display when `metrics/output/benchmark.png` exists.

The local dashboard URL is:

```
http://127.0.0.1:8000/static/index.html
```

Repository Layout

benchmarks/	Benchmark runner and report utilities
cli/	Command line interface
convera_core_api/	Public API boundary
convera_store/	Tensor store facade
convera_store_lite/	Fixed-size public chunk store
core/	Backend and model loading
inference/	Engine, KV manager, token graph
metrics/	Public repeated-prompt benchmark
models/	Local model directory
scripts/	Model download helpers
telemetry/	Metrics-only payload and client
tests/	Unit tests
ui/	FastAPI app and local dashboard
Manuals/	Project manuals and PDFs
audit/	Public audit trace helpers

Data Paths

Runtime paths are defined in config.py.

models/llama3	Default model path
data/chunks	Stored chunk payloads
data/index/kv_index.json	KV mapping index
data/index/model_index.json	Model tensor mapping index
data/index/token_graph.json	Token graph state
data/metrics_history.json	UI metrics history
data/learning_history.json	Public adaptive outcome history
data/routing_model.json	Public routing model state
data/execution_records.json	Public execution inspector records
data/audit_logs	Optional redacted audit traces

Features

- Local Hugging Face model execution.
- Prompt-level KV persistence.
- Public cache hit reporting.
- Local reusable tensor storage.
- Fixed-size chunk deduplication.
- Backend selection across CPU and supported accelerators.
- CLI command surface.
- Local FastAPI dashboard.
- Benchmark report tooling.
- Public repeated-prompt benchmark.
- Public benchmark graph generation.
- Benchmark graph endpoint for the UI.
- Public runtime precision selection.
- Deterministic public execution routing.
- Lightweight validation records.
- Public adaptive routing model.

- Execution inspector.
- Optional audit trace export.
- Metrics-only telemetry contract.
- Open-core boundary documentation.

Security And Privacy Position

The public project is designed to avoid exposing private runtime mechanisms. Public telemetry is metrics-only. Public benchmarks do not expose token-level traces or cache contents. The public repository uses neutral names and keeps non-public research systems out of the tree.

Current Limitations

- A compatible local model must already exist in models/llama3, or be downloaded with the provided helper.
- Some Hugging Face models require license acceptance and authentication.
- The public runtime is a lite implementation, not a production serving stack.
- Generated PDFs are local artifacts and are ignored by default unless force-added to git.

Recommended Next Steps

- Run the public repeated-prompt benchmark against an available local model.
- Capture UI screenshots for GitHub presentation.
- Build the server telemetry receiver from SERVER_HAND_OFF.md.
- Keep public changes focused on usability, metrics, and documentation.